

Resumo Completo: Processos e Threads (Sistemas Operacionais Modernos)

1.0 Processos

O conceito mais central de um sistema operacional é o **processo**, definido como **uma abstração de um programa em execução**. Ele representa a atividade de um programa, incluindo seu estado atual, recursos alocados e fluxo de execução.

1.1 O Modelo de Processo

- **Definição:** Um processo é uma entidade ativa que possui:
 - Um **espaço de endereçamento** (imagem de memória) contendo o código do programa, os dados e a pilha.
 - Um conjunto de **registradores**, incluindo o **contador de programa (PC)**, o **ponteiro de pilha (stack pointer)** e outros registradores de uso geral.
 - Um conjunto de **recursos** associados, como arquivos abertos, conexões de rede, etc.
- **Processo vs. Programa:** Um programa é uma entidade passiva (um arquivo em disco), enquanto um processo é uma entidade ativa e dinâmica. A mesma analogia do livro é útil: a receita de um bolo é o programa; o cozinheiro é o processador (CPU); os ingredientes são os dados; e o ato de cozinhar (ler a receita, pegar ingredientes, etc.) é o processo.
- **Multiprogramação e Pseudoparalelismo:** Em um sistema com uma única CPU, o sistema operacional alterna rapidamente a execução entre múltiplos processos. Embora em qualquer instante apenas um processo esteja de fato executando, a rápida alternância cria a **ilusão de paralelismo**, chamada de **pseudoparalelismo**. Em sistemas com múltiplos núcleos (multicore), o **paralelismo verdadeiro** ocorre, onde múltiplos processos executam simultaneamente.

1.2 Criação e Término de Processos

Eventos que levam à criação de processos:

1. **Inicialização do Sistema:** Quando o SO é carregado, vários processos de sistema (em primeiro plano) e processos de segundo plano (chamados de **daemons** em UNIX ou **serviços** em Windows) são criados.
2. **Chamada de Sistema por um Processo:** Um processo em execução pode criar novos processos para auxiliá-lo.
3. **Solicitação de um Usuário:** O usuário inicia um programa através da linha de comando ou de uma interface gráfica.
4. **Início de uma Tarefa em Lote:** Em sistemas de grande porte, um gerenciador de tarefas em lote pode iniciar o próximo trabalho da fila.

Mecanismos de Criação:

- **UNIX:** Utiliza a chamada de sistema `fork()`, que cria um clone exato do processo pai. O processo filho geralmente executa a chamada `execve()` em seguida para carregar um novo programa em seu espaço de endereçamento.
- **Windows:** Utiliza uma única função, `CreateProcess()`, que lida tanto com a criação do processo quanto com o carregamento do programa correto.

Término de Processos:

1. **Saída Normal (Voluntária):** O processo conclui seu trabalho e chama `exit()` (UNIX) ou `ExitProcess()` (Windows).
2. **Saída por Erro (Voluntária):** O processo detecta um erro (ex: arquivo não encontrado) e termina.
3. **Erro Fatal (Involuntária):** Causado por um erro de programa, como uma instrução ilegal, referência a memória inválida ou divisão por zero.
4. **Morte por Outro Processo (Involuntária):** Um processo executa uma chamada de sistema (`kill()` em UNIX, `TerminateProcess()` em Windows) para terminar outro processo.

1.3 Hierarquias de Processos

- **UNIX:** Processos formam uma árvore hierárquica. Um processo tem apenas um pai. O processo `init` é a raiz de toda a árvore. Processos e seus descendentes formam um **grupo de processos**.
- **Windows:** Não possui um conceito de hierarquia de processos. Todos os processos são iguais. Quando um processo cria outro, ele recebe um **handle** (identificador especial) que pode usar para controlar o filho, mas esse handle pode ser passado para outros processos, quebrando a noção de hierarquia.

1.4 Estados de um Processo

Um processo pode estar em um dos três estados fundamentais:

1. **Em Execução (Running):** Utilizando a CPU naquele instante.
2. **Pronto (Ready):** Executável, mas temporariamente parado para dar a vez a outro processo.
3. **Bloqueado (Blocked):** Incapaz de executar até que um evento externo ocorra (ex: conclusão de uma operação de E/S).

Transições entre estados:

- **Execução → Bloqueado:** O processo solicita algo que o força a esperar (ex: leitura de disco).
- **Execução → Pronto:** O escalonador decide que o processo já executou por tempo suficiente (preempção).
- **Pronto → Execução:** O escalonador escolhe este processo para ser o próximo a executar.
- **Bloqueado → Pronto:** O evento externo pelo qual o processo esperava ocorreu (ex: dados do disco chegaram).

1.5 Implementação de Processos

O sistema operacional mantém uma estrutura de dados chamada **tabela de processos** (ou **Bloco de Controle de Processo - PCB**), que é um array de estruturas, uma para cada processo. Cada entrada armazena informações vitais sobre o processo, incluindo:

- **Gerenciamento do Processo:** Registradores, contador de programa, palavra de estado do programa (PSW), ponteiro de pilha, estado do processo, prioridade, ID do processo (PID).
 - **Gerenciamento de Memória:** Ponteiros para os segmentos de texto, dados e pilha.
 - **Gerenciamento de Arquivos:** Diretório raiz, diretório de trabalho, descritores de arquivo, ID de usuário e grupo.
-

2.0 Threads

Threads são uma abstração que permite múltiplas execuções dentro do contexto de um único processo. São frequentemente chamados de **processos leves** (*lightweight processes*).

2.1 Utilização de Threads

Principais razões para usar threads:

1. **Modelo de Programação Simplificado:** Muitas aplicações realizam múltiplas atividades concorrentes. Decompor a aplicação em threads sequenciais que cooperam simplifica o design. Exemplo: um processador de texto com um thread para a interface do usuário, um para a formatação do texto em segundo plano e um para o salvamento automático.
2. **Desempenho:** Threads são mais "leves" (rápidos para criar e destruir) do que processos. Em sistemas com E/S substancial, threads permitem que a computação e a E/S se sobreponham, acelerando a aplicação.
3. **Paralelismo Real:** Em sistemas multicore, threads de um mesmo processo podem executar em paralelo em diferentes núcleos.

2.2 O Modelo de Thread Clássico

- **Compartilhamento de Recursos:** Todos os threads dentro de um mesmo processo compartilham:
 - O espaço de endereçamento (código e dados).
 - Variáveis globais.
 - Arquivos abertos.
- **Recursos por Thread:** Cada thread tem seus próprios e exclusivos:
 - **Contador de Programa (PC):** Para saber qual instrução executar.
 - **Registradores:** Para manter suas variáveis de trabalho atuais.
 - **Pilha (Stack):** Contendo o histórico de execução (chamadas de procedimento não retornadas).
 - **Estado** (Em execução, pronto, bloqueado).

2.3 Threads POSIX (Pthreads)

É um padrão (IEEE 1003.1c) para threads, amplamente suportado em sistemas UNIX-like. Funções principais:

- `pthread_create`: Cria um novo thread.
- `pthread_exit`: Termina o thread que a chamou.
- `pthread_join`: Bloqueia o thread chamador até que um thread específico termine.
- `pthread_yield`: Permite que um thread abra mão voluntariamente da CPU.

2.4 Implementação de Threads

1. Threads em Espaço de Usuário:

- **Como funciona:** O pacote de threads é implementado como uma biblioteca no espaço do usuário. O núcleo não tem conhecimento da existência de threads; ele enxerga apenas um processo de thread único.
- **Vantagens:**

- Troca de threads extremamente rápida (não requer uma chamada de sistema).
- Pode ser implementado em sistemas operacionais que não suportam threads nativamente.
- Cada processo pode ter seu próprio algoritmo de escalonamento de threads.
- **Desvantagens:**
 - Se um thread faz uma chamada de sistema bloqueante (ex: **read**), o **processo inteiro** é bloqueado.
 - Uma falta de página bloqueia o processo inteiro.
 - Não há como explorar o paralelismo multicore.

2. Threads em Espaço de Núcleo (Kernel):

- **Como funciona:** O núcleo gerencia todos os threads. A tabela de threads fica dentro do núcleo. Chamadas para criar ou destruir threads são chamadas de sistema.
- **Vantagens:**
 - Se um thread bloqueia, o núcleo pode escalonar outro thread do mesmo processo (ou de outro).
 - Em sistemas multicore, o núcleo pode escalonar threads do mesmo processo em múltiplos núcleos simultaneamente.
- **Desvantagens:**
 - A criação, destruição e troca de threads têm um custo significativamente maior, pois exigem uma chamada de sistema.

3. Implementações Híbridas:

- Buscam combinar o melhor dos dois mundos, multiplexando um número de threads de usuário em um número menor (ou igual) de threads de núcleo. Um exemplo é o modelo de **ativações pelo escalonador**.

3.0 Comunicação e Sincronização Entre Processos (IPC)

Processos (e threads) que cooperam precisam de mecanismos para se comunicar e sincronizar suas ações para evitar o caos.

3.1 Condições de Corrida (Race Conditions)

Ocorrem quando dois ou mais processos acessam dados compartilhados e o resultado final depende da ordem precisa em que as operações de acesso ocorrem. O exemplo clássico é o **spooler de impressão**, onde dois processos podem tentar pegar a mesma vaga livre no diretório de spool, resultando na perda de um dos trabalhos de impressão.

3.2 Regiões Críticas e Exclusão Mútua

- **Região Crítica:** A parte do código onde um processo acessa recursos compartilhados (memória, arquivos, etc.).
- **Exclusão Mútua:** Um mecanismo para garantir que, se um processo está executando em sua região crítica, nenhum outro processo possa entrar na sua própria região crítica (para o mesmo recurso compartilhado).

Condições para uma boa solução:

1. Apenas um processo pode estar na região crítica por vez.
2. Nenhuma suposição pode ser feita sobre a velocidade ou o número de CPUs.
3. Nenhum processo executando fora de sua região crítica pode bloquear outros processos.
4. Nenhum processo deve esperar eternamente para entrar em sua região crítica.

3.3 Soluções para Exclusão Mútua

Com Espera Ocupada (Busy Waiting):

- **Desabilitar Interrupções:** Simples, mas perigoso para processos de usuário. Não funciona em sistemas multicore, pois só afeta a CPU que executou a instrução.
- **Variáveis de Trava:** Uma variável compartilhada é testada. Falha devido a uma condição de corrida no próprio teste e configuração da trava.
- **Alternância Estrita:** Viola a condição 3, pois força os processos a alternarem estritamente, mesmo que um seja muito mais lento ou não queira entrar na região crítica.
- **Solução de Peterson:** Uma solução de software correta que combina a ideia de alternância (**turn**) com a de interesse (**interested** array).
- **Instruções de Hardware (TSL/XCHG):** A instrução **TSL** (Test and Set Lock) lê e modifica uma palavra de memória de forma **atômica** (indivisível). **XCHG** (Exchange) troca o conteúdo de um registrador e memória atômicamente. São a base para implementações mais complexas.

Com Bloqueio (Sem Espera Ocupada):

- **Sleep e Wakeup:** Primitivas onde **sleep** bloqueia o processo e **wakeup** o acorda. Sofre do **problema do sinal de despertar perdido** (*lost wakeup problem*), uma condição de corrida sutil onde um **wakeup** é enviado a um processo que ainda não foi dormir.
- **Semáforos (Dijkstra):**
 - Uma variável inteira que evita sinais perdidos.
 - Operações atômicas:
 - **down()** (ou **P**): Decrementa o valor. Se o valor se torna negativo, o processo bloqueia.
 - **up()** (ou **V**): Incrementa o valor. Se houver processos bloqueados no semáforo, um deles é acordado.
 - **Semáforo Binário:** Restrito aos valores 0 e 1, usado para exclusão mútua.
- **Mutexes:**
 - Uma versão simplificada do semáforo binário, usada exclusivamente para exclusão mútua. Possui dois estados: travado (locked) e destravado (unlocked).
 - **Futex (Fast Userspace Mutex):** Uma implementação híbrida em Linux que evita a chamada de sistema se não houver contenção pela trava.
- **Monitores (Hoare e Brinch Hansen):**
 - Um conceito de alto nível, presente em algumas linguagens de programação (como Java com **synchronized**).
 - É um conjunto de rotinas, variáveis e estruturas de dados onde o compilador garante a exclusão mútua automaticamente.
 - Usa **variáveis de condição** com as operações **wait** (bloqueia) e **signal** (acorda um processo em espera).
- **Troca de Mensagens:**

- Usada para comunicação entre processos em máquinas diferentes ou com espaços de endereçamento separados.
 - Primitivas: `send(destination, &message)` e `receive(source, &message)`.
 - **Barreiras:**
 - Mecanismo de sincronização para um grupo de processos. Nenhum processo pode passar da barreira até que todos os processos do grupo a tenham alcançado.
-

4.0 Escalonamento

O **escalonador** é a parte do SO que decide qual processo pronto deve ser alocado para a CPU. O algoritmo que ele utiliza é o **algoritmo de escalonamento**.

4.1 Conceitos de Escalonamento

- **Processos Limitados pela CPU vs. Limitados pela E/S:**
 - **Limitados pela CPU:** Passam a maior parte do tempo computando (surtos longos de CPU).
 - **Limitados pela E/S:** Passam a maior parte do tempo esperando por E/S (surtos curtos de CPU).
- **Preemptivo vs. Não Preemptivo:**
 - **Não Preemptivo:** Um processo executa até bloquear ou liberar voluntariamente a CPU.
 - **Preemptivo:** Um processo executa por um tempo máximo (quantum). Se ainda estiver executando ao final do quantum, ele é suspenso, e o escalonador escolhe outro.
- **Metas do Escalonamento:** Variam com o ambiente (lote, interativo, tempo real), mas incluem justiça, eficiência, tempo de resposta, tempo de retorno e vazão.

4.2 Algoritmos de Escalonamento

Para Sistemas em Lote (Batch):

- **Primeiro a Chegar, Primeiro a ser Servido (FCFS):** Simples, mas um processo longo pode fazer processos curtos esperarem muito.
- **Tarefa Mais Curta Primeiro (SJF):** Não preemptivo. É otimizado para o tempo de retorno médio, mas requer conhecimento prévio do tempo de execução.
- **Tempo Restante Mais Curto em Seguida (SRTN):** Versão preemptiva do SJF.

Para Sistemas Interativos:

- **Chaveamento Circular (Round-Robin):** O mais comum. Cada processo recebe um quantum de tempo. A escolha do tamanho do quantum é crucial (pequeno demais = alto overhead; grande demais = tempo de resposta ruim).
- **Escalonamento por Prioridades:** Processos com maior prioridade executam primeiro. Risco de **inanição** (*starvation*) para processos de baixa prioridade. Pode usar prioridades dinâmicas para mitigar isso (ex: `aging`).
- **Múltiplas Filas:** Múltiplas filas de processos prontos, uma para cada nível de prioridade.
- **Escalonamento por Loteria:** Processos recebem "bilhetes" para recursos. Sorteios determinam quem executa. Processos mais importantes recebem mais bilhetes.
- **Fração Justa (Fair-Share):** Leva em conta o dono do processo para garantir que cada usuário receba sua fração justa da CPU, independentemente de quantos processos ele execute.

4.3 Escalonamento de Threads

- **Threads em Espaço de Usuário:** O escalonador do SO escolhe um processo (ex: A). O escalonador de threads dentro do processo A decide qual thread (ex: A1, A2) executar. A decisão é local ao processo.
- **Threads em Espaço de Núcleo:** O escalonador do SO escolhe diretamente qual thread executar de um conjunto global de threads. Ele pode ser mais inteligente, por exemplo, preferindo escalonar um thread do mesmo processo que acabou de executar para reduzir o custo de troca de contexto (evitando a invalidação de cache de memória).